Roteiro do vídeo-relatório — até 10 minutos

Base: notebook executado, guia revisado, matriz e fontes do repositório. Duração planejada: **9min40s**, com 20 segundos de margem. As falas são uma base para ensaio; o tempo real depende do ritmo e das transições.

Substitua `[Nome 1]` e `[Nome 2]`. A distribuição de responsabilidades abaixo é uma **proposta realista**, não um registro histórico comprovado. Ajustem antes de gravar para refletir o que cada pessoa realmente fez. Não atribuam a alguém trabalho que ela não realizou.

Sequência e tempo

Slide	Intervalo	Tema	Apresentador sugerido
1	0:00–0:35	Problema e objetivo	Integrante 1
2	0:35–1:25	O que foi desenvolvido e como	Integrante 1
3	1:25–2:10	Topologia e perfis de rede	Integrante 2
4	2:10–3:00	Matriz e método de análise	Integrante 2
5	3:00–3:45	Demonstração e cobertura	Integrante 1
6	3:45–4:45	Resultados de tempo de resposta	Integrante 2
7	4:45–5:40	Resultados de vazão individual	Integrante 1
8	5:40–6:20	HEAD e significado dos resultados	Integrante 1
9	6:20–7:25	Integridade, calibração e limites	Integrante 2
10	7:25–8:20	Contribuições dos dois integrantes	Ambos
11	8:20–9:40	Autoavaliação e conclusão	Ambos

Slide 1 — Problema e objetivo

Adicionar ao slide: título “Transferência concorrente de arquivos: desempenho em redes emuladas”; nomes dos dois integrantes; disciplina; uma pergunta central: “Como rede, tamanho e concorrência afetam tempo e vazão?”; ícones de cliente e servidor.

Fala sugerida — Integrante 1:

> Nosso trabalho desenvolveu uma aplicação cliente-servidor de arquivos em C e analisou seu desempenho em diferentes condições de rede. O objetivo foi entender como o tamanho do arquivo, a operação e a quantidade de clientes concorrentes influenciam o tempo de resposta e a vazão útil. Escolhemos esse problema porque ele reúne programação com sockets e avaliação experimental de um serviço semelhante aos usados para compartilhar fotos e vídeos.

Slide 2 — O que foi desenvolvido e como

Adicionar ao slide: diagrama `Cliente C → TCP / subconjunto de HTTP → Servidor C`; três linhas: GET baixa, PUT envia, HEAD consulta metadados; pequenos rótulos “thread por conexão”, “temporário exclusivo” e “publicação com rename”. No rodapé: `src/client.c`, `src/server.c`, `src/protocol.c`.

Fala sugerida — Integrante 1:

> Implementamos cliente e servidor com a API de sockets POSIX em C. O protocolo é um subconjunto controlado de HTTP sobre TCP, com uma operação por conexão. GET recebe um arquivo, PUT envia um arquivo ao servidor e HEAD consulta os metadados sem transferir seu conteúdo. O servidor cria uma thread por conexão para permitir atendimento concorrente. A implementação trata mensagens parciais e usa arquivos temporários exclusivos antes da publicação do destino. Isso reduz o risco de deixar um arquivo parcialmente transferido como resultado final. HEAD foi incluído como extensão conservadora do grupo, considerando a ambiguidade apontada no guia.

Orientação visual: apresentar o fluxo e as decisões; evitar uma captura de código com dezenas de linhas. Não afirmar conformidade completa com HTTP.

Slide 3 — Onde: topologia e perfis

Adicionar ao slide: captura legível da sessão CORE salva ou desenho fiel `Cliente → acesso → núcleo → gargalo → núcleo → acesso → Servidor`; cliente `10.10.1.10`, servidor `10.10.5.10`; destacar `R_Nucleo_1 ↔ R_Nucleo_2`. Tabela:

Perfil	Capacidade nominal	Atraso configurado por sentido	Perda configurada
A	100 Mbit/s	5 ms	0%
B	20 Mbit/s	20 ms	0%

Fala sugerida — Integrante 2:

> A topologia salva no CORE coloca cliente e servidor em redes de acesso distintas, conectadas por um núcleo. O enlace entre os dois roteadores de núcleo representa o gargalo. Comparamos o perfil A, de 100 megabits por segundo e 5 milissegundos de atraso por sentido, ao perfil B, de 20 megabits e 20 milissegundos. Não configuramos perda. Os perfis representam condições simplificadas de acesso fixo e móvel, e não características universais dessas tecnologias. Esses valores são parâmetros configurados; a comprovação da calibração é uma limitação que retomaremos.

Conferência: os XMLs de A/B preservam `delay=5000/20000` e `bandwidth=100000000/20000000`. A descrição antiga `topologia-etapa4.txt` usa “latência 10 ms” sem a mesma precisão por sentido; use os valores dos XMLs e da matriz, sem chamá-los de RTT medido.

Slide 4 — Como os experimentos foram analisados

Adicionar ao slide: matriz visual `2 perfis × 3 operações × 2 tamanhos × 3 concorrências = 36 células`; tamanhos 2/10 MiB; concorrências 1/8/32; 11 rodadas por célula; total de 396 rodadas e 5.412 requisições. Caixa em destaque: “Unidade do IC: rodada completa”.

Fala sugerida — Integrante 2:

> A matriz combina os dois perfis, três operações, arquivos de 2 e 10 mebibytes e concorrências de 1, 8 e 32. Os tamanhos representam cargas de compartilhamento de fotos e vídeos curtos. São 36 configurações, com 11 rodadas cada, totalizando 396 rodadas e 5.412 requisições. O notebook lê os JSONs brutos e confere os fatores contra a matriz. Para a inferência, calculamos médias por rodada e depois média, mediana, desvio amostral e intervalo de confiança de 95% com Student. Workers da mesma rodada compartilham recursos e não são tratados como repetições independentes.

Slide 5 — Demonstração e cobertura dos dados

Adicionar ao slide: clipe de aproximadamente 15 segundos mostrando uma execução real de GET e PUT com os executáveis C e sua saída JSON; ao lado, resumo do notebook: 36/36 células com 11 rodadas completas; zero requisições ausentes; zero falhas detectadas nos critérios avaliados. Rodapé: “Integridade de PUT no destino: não comprovada pelos hashes preservados”.

Fala sugerida — Integrante 1:

> Aqui mostramos a interface do cliente e a saída estruturada que registra código de resposta, bytes, duração e vazão. Na campanha analisada, todas as 36 configurações têm as 11 rodadas completas previstas. Não encontramos requisições ausentes nem falhas nos critérios de telemetria avaliados, e os hashes preservados de GET não apresentaram divergências. Isso demonstra cobertura e consistência dos registros analisados, mas não comprova todas as propriedades do sistema: no PUT, falta o hash do arquivo publicado no servidor.

Preparação: gravar o clipe antes do vídeo, em ambiente disponível e identificado. Caso a demonstração seja em localhost, rotular “demonstração funcional em localhost”; ela não representa evidência da topologia CORE. Se não houver nova execução, mostrar registros existentes com a legenda “registro da campanha”, sem simular uma execução ao vivo. Não mostrar comandos digitados sem tempo para sua execução real. Teste de sobrescrita só deve ser anunciado como demonstrado se houver evidência antes/depois.

Slide 6 — Tempo de resposta: efeito da rede e da concorrência

Adicionar ao slide: usar os painéis GET A/B do gráfico de tempo do notebook, ampliados; não colocar a figura inteira com seis painéis em tamanho ilegível. Destacar o arquivo de 10 MiB:

Concorrência	Perfil A	Perfil B
1	960,8 ms	4.729,8 ms
32	28.105,4 ms	137.936,0 ms

Legenda: “Médias entre rodadas; barras = IC95% de Student; n=11 por célula”.

Fala sugerida — Integrante 2:

> O primeiro resultado é o aumento do tempo com a concorrência, especialmente nos arquivos maiores. No GET de 10 mebibytes, com um cliente, a média ficou próxima de 961 milissegundos no perfil A e 4,73 segundos no perfil B. Com 32 clientes, passou para aproximadamente 28,1 e 137,9 segundos. O perfil B teve maior latência nas 18 comparações equivalentes de método, tamanho e concorrência. As barras representam a incerteza entre rodadas, não entre clientes da mesma rodada. O comportamento

> é compatível com maior tempo de serviço e compartilhamento de recursos. Como capacidade e atraso mudam juntos entre A e B, não podemos atribuir toda a diferença isoladamente à latência ou à banda.

Slide 7 — Vazão útil individual

Adicionar ao slide: painéis GET A/B do gráfico de vazão individual do notebook; destacar 10 MiB e esta tabela:

Concorrência	Perfil A	Perfil B
1	87,322 Mbit/s	17,744 Mbit/s
32	2,997 Mbit/s	0,609 Mbit/s

Adicionar a fórmula `vazão individual = bytes úteis × 8 / tempo em segundos / 10⁶`. Não usar o gráfico antigo de HEAD nem o campo antigo `throughput_aggregate_mbps` como vazão medida do enlace.

Fala sugerida — Integrante 1:

> A vazão aqui é a taxa útil de cada transferência, calculada a partir dos bytes do arquivo e de sua duração. No GET de 10 mebibytes com um cliente, obtivemos aproximadamente 87,3 megabits por segundo em A e 17,7 em B. Com 32 clientes, a média individual caiu para cerca de 3 e 0,61 megabit por segundo. O perfil B teve menor vazão individual nas 12 comparações equivalentes de GET e PUT. A redução por cliente é coerente com o compartilhamento dos recursos disponíveis. Contudo, esses valores não são a vazão agregada da rodada; a queda individual, sozinha, não demonstra uma queda da capacidade total do serviço.

Slide 8 — GET, PUT e HEAD

Adicionar ao slide: três cartões: GET transfere servidor→cliente; PUT transfere cliente→servidor; HEAD recebe metadados. Exemplo com concorrência 1:

Operação e tamanho	Tempo A	Tempo B
GET, 10 MiB	960,8 ms	4.729,8 ms
PUT, 10 MiB	1.171,9 ms	4.814,3 ms
HEAD, referência 2 MiB	22,1 ms	83,2 ms

Fala sugerida — Integrante 1:

> GET e PUT transferem conteúdo em sentidos opostos e também envolvem custos diferentes de leitura, escrita e confirmação. Com um cliente e arquivo de 10 mebibytes, o PUT levou cerca de 1,17 segundo em A e 4,81 segundos em B. Já o HEAD sobre o arquivo de referência de 2 mebibytes levou aproximadamente 22 e 83 milissegundos. Como HEAD não envia o corpo, seu tamanho de referência não representa bytes transferidos. Por isso, analisamos sua latência e mantemos sua vazão de conteúdo como não aplicável. Esses tempos também não devem ser interpretados como medições de RTT por ping.

Slide 9 — O que podemos concluir e quais são os limites

Adicionar ao slide: duas colunas, “Evidências disponíveis” e “Pendências”. Na primeira: cobertura completa, separação dos fatores, hashes GET compatíveis, IC entre rodadas. Na segunda: hash do destino PUT; logs de calibração por sentido; janela comum da rodada. Uma frase destacada: “Compartilhamento observado; ponto de saturação não determinado”.

Fala sugerida — Integrante 2:

> A análise identificou limitações relevantes. Primeiro, os hashes de GET permitem comparar entrada e saída, enquanto os registros de PUT preservam apenas o hash da entrada. Portanto, não declaramos integridade comprovada do destino para toda a campanha. Segundo, os arquivos disponíveis de calibração descrevem a topologia, mas não preservam os logs de ping, traceroute e iperf necessários para verificar os dois sentidos. Terceiro, falta uma janela comum medida da rodada. A análise anterior somava vazões individuais, o que não equivale a dividir bytes totais pela duração conjunta. O notebook deixa a vazão agregada indisponível. Podemos descrever aumento de tempo e queda da taxa individual, mas não estabelecer com rigor o ponto de saturação nem atribuí-lo exclusivamente ao enlace. Uma nova coleta deve preservar essas evidências e acompanhar recursos do host.

Slide 10 — Contribuições individuais

Adicionar ao slide: nomes e responsabilidades em duas colunas. Usar esta divisão somente se representar a participação real:

Responsabilidade	Integrante 1 — foco em aplicação	Integrante 2 — foco experimental
Implementação principal	Protocolo, servidor concorrente e tratamento de arquivos	Cliente, telemetria e automação de coleta
Verificação	Casos de GET/PUT, erros, interrupções e sobrescrita	Matriz, consistência dos registros e evidências da topologia
Análise e comunicação	Revisão das métricas e interpretação do comportamento da aplicação	Notebook, estatística por rodada, gráficos e organização dos resultados
Atividade conjunta	Integração, revisão cruzada e gravação	Integração, revisão cruzada e gravação

Fala sugerida — Integrante 1, adaptar à realidade:

> Minha contribuição se concentrou no protocolo e no servidor, incluindo o atendimento concorrente e o tratamento dos arquivos. Também participei da integração com o cliente e da revisão do comportamento de GET, PUT e dos erros. Na discussão dos resultados, revisei como as decisões de implementação poderiam influenciar os tempos observados.

Fala sugerida — Integrante 2, adaptar à realidade:

> Minha contribuição se concentrou no cliente e na telemetria, na organização da matriz e na automação da coleta. Também organizei a análise em Python, os gráficos e as verificações de consistência. Trabalhamos juntos na integração e na interpretação dos resultados, e identificamos as evidências que ainda precisam ser melhoradas.

Ajuste obrigatório: se a mesma pessoa fez cliente e servidor, ou se o notebook foi produzido com assistência de ferramenta, redistribuir a descrição com honestidade, conforme a política da disciplina. Não dizer que

todos os testes ou a calibração foram concluídos apenas por constarem como responsabilidades propostas.

Slide 11 — Autoavaliação e conclusão

Adicionar ao slide: três blocos curtos:

- **Avanços:** aplicação C, campanha com cobertura da matriz, análise por rodada e reconhecimento das limitações.
- **A melhorar:** evidência de calibração, hash remoto de PUT e janela comum para goodput agregado.
- **Aprendizado:** distinguir implementação funcional, resultado registrado e conclusão experimental sustentada.

Rodapé: caminhos do notebook e das fontes; link do repositório se desejado. Evitar uma nota numérica sem rubrica do professor.

Fala sugerida — Integrante 1:

> Nossa autoavaliação é positiva quanto à integração entre a aplicação em C e a análise dos dados. Conseguimos organizar uma campanha com as combinações previstas e observar de forma consistente que o perfil B apresentou maiores tempos e menores vazões individuais. Também entendemos melhor o efeito do tamanho e da concorrência sobre o serviço. Consideramos um avanço importante analisar as rodadas corretamente e revisar as métricas anteriores quando encontramos uma interpretação inadequada.

Fala sugerida — Integrante 2:

> Ao mesmo tempo, avaliamos que a documentação experimental precisa melhorar. Faltam evidências de calibração por sentido, a verificação do destino de PUT e uma janela comum para calcular a vazão agregada. Por isso, não consideramos demonstrados todos os pontos da avaliação, especialmente a saturação do enlace. Nossa prioridade seria corrigir a coleta e repetir as medições necessárias, preservando os dados existentes. O principal aprendizado foi que desenvolver o sistema e medir seu desempenho são etapas complementares: a conclusão precisa respeitar a evidência disponível.

Frase final — Integrante 2:

> O trabalho mostrou o impacto da rede e da concorrência no desempenho individual das transferências e deixou um caminho claro para melhorar a validação experimental. Obrigado.

Preparação da gravação

1. Ensaiar com cronômetro; alvo de 9min40s, incluindo troca de apresentador e clipe. Se exceder, reduzir detalhes de arquitetura antes de cortar contribuições ou limitações.
2. Usar os gráficos incorporados ao notebook atual; não os gráficos antigos em `analysis/figures/` sem revisão. Em cada slide de resultados, mostrar apenas os painéis necessários, com unidades, fatores, n e significado das barras legíveis.
3. Os números das tabelas são arredondamentos dos resultados da campanha ativa `experiments/raw`; os dois perfis são comparados mantendo método, tamanho e concorrência iguais. Não misturar backups para aumentar n.
4. Diferenciar a demonstração gravada, os registros da campanha e os parâmetros configurados. Não anunciar que a calibração está aprovada ou que o sistema tem zero falhas em qualquer situação.

5. Identificar corretamente as contribuições. A autoavaliação sugerida reconhece avanços e pendências; não afirma aprovação integral dos requisitos acadêmicos.
6. Conferir a duração final e o acesso ao link do vídeo. Conforme o enunciado, a entrega inclui código(s) do servidor e link do vídeo de até 10 minutos.